Apuntes File System - clase de power point

Filmina 1) File System - Gestión de la Información y Archivos

F2) Agenda

Archivo (persistencia de la información).
Sistema de **Gestión de Archivos**, ¿A que nos referimos cuando hablamos del sistema de gestión de Archivos?
Sistema de archivos en **sentido amplio** y luego en **sentido estricto**.
Volveremos a la definición de sistemas de archivos en sentido amplio para detenernos en algunas de las **convenciones de diseño** más significativas: el nombrado, estructura, tipos, atributos, operaciones, la organización lógica de información, modos de acceso a los archivos. Los métodos de asignación de archivos. y la gestión del espacio libre.

F3) Archivos
Ya hemos hablado y hemos repasado el concepto de archivo, como conjunto de bits o de bytes, lo que vamos a rescatar aquí es que, el archivo es la unidad de representación de la información en un sistema de computación. Todo aquello que sea información en un sistema de computación tiene forma de archivo, es el archivo esa unidad de representación.
Es el medio para concretar la persistencia de la información (permite el guardado o resguardo de la información y su posterior recuperación, incluso a posteriori de que el equipo haya sido apagado). A diferencia del alojamiento en memoria principal, donde la información tiene carácter de volátil, en los archivos tiene carácter de persistente.
Un archivo es tratado como una entidad única y está identificado por un nombre o File-Id unívoco. (es decir, no pueden haber dos archivos con el mismo nombre dentro de un sistema de computación, al menos con el mismo **nombre completo**, esto significa su nombre propio más su ubicación relativa dentro del esquema de directorios del sistema de computación).
Un archivo es un conjunto de bits en forma lineal o bien un conjunto de registros homogéneos(todos ellos con igual estructura) (compuestos por campos, como cualquier estructura de tipo registro).

F4) Sistema de Gestión de Archivos
//A la información representada en forma de archivos hay que gestionarla, y obviamente es responsabilidad del sistema operativo gestionar la información.
El sistema de gestión de archivos es un módulo del sistema operativo. //que se encarga de la administración o de la gestión de la información.
Permite que un usuario o un proceso puedan acceder a los archivos y manipularlos.
//Genera esta interfaz, de la que generalmente hablamos, entre el hardware y el

usuario, en este caso para permitirle manipular la información en formato de archivos.
Brinda una interfaz que evita que el programador tenga que desarrollar software específico para interactuar con sus archivos. //Significa que el programador se despreocupa del archivo, de como está alojado el archivo, de su estructura interna, y es el sistema operativo quien gestiona eso, y le facilita la vida al desarrollador y obviamente a los usuarios que utilizan la información.
Garantiza la corrección de los datos y la estructura de los archivos. //Conoce cómo manipular los archivos, y debe garantizar el correcto resguardo y la correcta recuperación de los datos. Conoce además la estructura de los archivos, cuando un proceso trabaja con un determinado archivo y ese archivo tiene una estructura determinada, es el sistema operativo el garante de que esa estructura se va a respetar y mantener correctamente.

F5) Sistema de gestión de archivos
Ofrece un conjunto estándar de rutinas de interfaz y operaciones de entrada y salida. //es decir es el sistema de gestión de archivos quien determina qué cosas puede hacer un usuario en forma directa o un proceso como intermediario del usuario respecto de los archivos.
Optimiza el rendimiento de las operaciones sobre archivos. //es decir, el sistema de gestión de archivos debe tratar de hacer un uso eficiente del recurso información, de forma tal que la eficiencia y la eficacia sea la mejor posible.
Ofrece soporte de entrada y salida para los diversos dispositivos del sistema de computación, garantizando la transparencia de su arquitectura (**Independencia lógica del dispositivo** //es un aspecto fundamental dentro de lo que es un sistema de gestión de archivo)

//"Ofrece soporte de entrada y salida para los diversos dispositivos" significa que cualquiera sea el dispositivo de almacenamiento de la información el mecanismo por el cual el sistema de gestión de archivo administra los archivos es el mismo y se abstrae de la arquitectura física del dispositivo. Para eso el driver que dialoga con el controlador se pondrán de acuerdo y conocerán de qué manera la información es guardada y recuperada (físicamente hablando) en función de la arquitectura del dispositivo. Pero para el usuario y para el desarrollador el dispositivo es transparente, nadie sabe si lo que tenemos como unidad C es efectivamente un disco rígido o un pendrive o un disco de estado sólido o un dvd. ni el usuario ni el desarrollador sabe cómo está almacenada la información.
Esto significa que el sistema de gestión de archivo provee una serie de actividades de operaciones que permiten al usuario en forma directa o al proceso interactuar con la información haciendo caso omiso a la arquitectura del dispositivo, esto se conoce como **independencia lógica del dispositivo**. Nos independizamos del dispositivo y lo vemos como un dispositivo lógico virtual.

Asume a los dispositivos como un conjunto de bloques. //en alguna medida un dispositivo se mapea como un conjunto de bloques de información. El **bloque** es la unidad mínima de lectura y escritura de información en un dispositivo. Ningún proceso ningún usuario puede leer ni escribir menos que un bloque.

F6) Sistema de gestión de archivos

Funciones

Entre las funciones de un sistema de gestión de archivos podemos mencionar las siguientes:

Identificar y ubicar el archivo en cuestión.

Utiliza un directorio que describe la ubicación de todos los archivos y sus atributos. //Todos los archivos en un dispositivo de almacenamiento están referenciados por una entrada maestra que se llama **directorio** que contiene los archivos y los atributos de cada uno de estos archivos.

Aplica el control de acceso a los usuarios. //Es el que discrimina que usuario puede acceder o no a cual archivo, y que privilegios tiene un usuario sobre un determinado archivo. El garante de esta protección (del uso adecuado de los archivos) es precisamente el sistema de gestión de archivos. Tiene la responsabilidad de controlar y verificar el acceso y los privilegios necesarios de los usuarios (por sí mismos o por intermedio de los procesos) a los archivos a los que están refiriendo.

Las **operaciones de entrada y salida se llevan a cabo por bloques**. //no se puede leer ni escribir una unidad menor que un bloque, en caso de querer leer o escribir una cantidad mayor a un bloque entonces se lee o se escribe un conjunto de bloques.

Asigna los archivos a los bloques disponibles. //es el que decide físicamente donde se van a ubicar los archivos dentro de un dispositivo a partir de conocer cuales son los bloques que tiene disponible para su utilización.

Gestiona el espacio libre, de manera que se conozca qué bloques están disponibles. //Y en función de eso, cuando necesita espacio para alojar un archivo recurre a utilizar o ocupar estos bloques libres. A partir de ese momento los bloques necesarios dejan de estar libres.

F7) Sistema de Archivos

Dos definiciones de File System, en sentido amplio y en sentido estricto.

Cuando hablamos de **File System, en sentido amplio**, nos referimos al **conjunto de convenciones** de diseño que determinan el manejo de la información en un SO.

Estas convenciones se establecen **en tiempo de diseño del producto**, luego se implementan y rigen la vida de los usuarios del SO en lo que respecta al manejo de la información.

//Significa que en tiempo de diseño, cuando todavía el sistema operativo no existe, el área de diseño del sistema de gestión de archivo decide cómo van a manejar la información, *concluyen en una serie de convenciones que documentan*, estas convenciones establecen como van a manipular la información. Posteriormente las convenciones pasan al área de desarrollo para tenerlas en cuenta a la hora de la construcción del SO. Los futuros usuarios del sistema operativo van a tener que respetar estas convenciones, porque signan la forma de operar con archivos dentro del SO en cuestión.

Cuando hablamos de “el **File System de Linux** o el File System de Windows” nos referimos al File System de un SO en sentido amplio. //es decir estamos hablando de las características que tiene el sistema de archivos de ese SO en forma genérica más allá de las instancias de ese SO.

//Podemos decir “El File System de windows admite más de un punto y admite nombres largos en cambio el File System de DOS admite solo nombres de archivos de 8 caracteres con una extensión de a lo sumo 3 caracteres una sola extensión y esta extensión además es opcional” estaríamos hablando sobre características que hacen a la gestión de la información en el sistema operativo, entonces ***estaríamos hablando del File System en sentido amplio*** como conjunto de convenciones que determinan la forma en la que el sistema operativo administra la información. Y por ende los usuarios debemos respetar.

F8) Sistema de Archivos

En cambio cuando hablamos del **File System, sentido estricto**, estamos hablando de:
El conjunto de información, estructurada bajo determinadas pautas, en un sistema de computación. Es decir, estamos hablando de una instancia. Es el conjunto de información de cada Sistema de computación. //El file system de mi máquina con un determinado SO, que puede ser el mismo SO que tiene mi vecino. Mi sistema operativo contiene la información personal mía que es diferente de la información que tendrá mi vecino. Mi información conforma mi Sistema de archivos. Mi información está estructurada en archivos según las pautas que me fija el SO.
Cuando hablamos de “el File System de mi PC” o “el File System de la pc de María” nos referimos al File System de un SO **en sentido estricto**. //Es decir el File System de un sistema de computación.

F9) Sistema de Archivos

Dos Sistemas de Computación con igual SO tienen iguales FS en sentido amplio, pero diferente en sentido estricto, dado que la información propiamente dicha que manejan es diferente, pero en ambos casos respetan las mismas convenciones de diseño, por tener el mismo SO.

//Es muy importante poder discernir cuando nos referimos a FS en sentido amplio y en sentido estricto.
//En sentido amplio es ese conjunto de decisiones que se tomaron cuando se diseñó el sistema operativo.
//En sentido estricto, estamos hablando de la información propiamente dicha, esa que podemos manejar en nuestro respectivo Sistema de Computación, sobre la que tenemos la posibilidad de operar.

F10) Convenciones de diseño

Volviendo entonces al concepto de File System en sentido amplio y a las convenciones de diseño. Las **convenciones de diseño** son las reglas o pautas para la manipulación de la información que el Sistema de Gestión de Archivos implementará y el usuario respetará. Refieren a:
Reglas de Nombrado como se denominan los archivos.
Estructura

Tipos
Atributos
Operaciones
Organización Lógica
Modos de acceso a los archivos
Métodos de asignación de archivos.
Estructuras para la gestión del espacio libre.
Criterio de selección del espacio libre.
//En estas reglas vamos a poner el énfasis en lo que hace a sistemas de gestión de archivos o file system en sentido amplio.

F11) Reglas de naming
Las reglas de "naming" o de nombrado establecen las pautas para la identificación de los archivos (identificación del archivo está dada por su nombre). //Tenemos que regular o pautar de qué manera se va a poder llamar a un archivo.
Hay que definir qué cantidad de caracteres se van a permitir en un nombre.
Hay que definir qué cantidad de caracteres se van a permitir en una extensión. //Tenemos que definir si vamos a permitir archivos de tipo "nombre.extensión". Si la extensión va a ser obligatoria o va a ser opcional.
Hay que definir cuales son los caracteres no permitidos. Definir también cuál va a ser el uso de los caracteres especiales, espacios y puntos.
También la cantidad de puntos es algo a definir.
¿Extensión obligatoria?
¿Case sensitive?
Otra regla que tenemos que definir es la Asociación de aplicaciones a extensiones, //si algunas extensiones podrán asociarse a alguna aplicación, que sería la habitualmente utilizada para manipular ese tipo de archivos.

F12) Estructura de archivos
Otro punto importante que tiene que ver con las reglas de diseño es la estructura de archivos que vamos a soportar en este SO.
En principio podríamos pensar que vamos a soportar archivos **no estructurados** (vamos a considerar que un archivo es una secuencia de bytes) sin un criterio o una lógica de ordenamiento dentro de estos bytes.
Otra opción es que aceptemos **archivos estructurados simples** dentro de estos, tenemos aquellos **archivos de línea**, donde tenemos una línea tras otra línea, y cada línea es una secuencia de bytes (conjunto de caracteres). Tengo una unidad de lectura y escritura que será la línea, esa línea no tiene estructura interna.
Otra opción sería los estructurados de largo fijo, donde yo tengo una cierta estructura dentro de cada línea, esa estructura tiene un campo de **largo fijo**. O bien de **largo variable** y se desconoce a priori el largo que va a tener la estructura. Pero si se cual es la estructura que va a tener, es

decir, van a ser estructurados.
En tanto a los **estructurados complejos** tenemos un archivo como un conjunto de registros. una **estructura con un formato definido por el usuario** homogénea, es decir, todos los registros van a tener el mismo formato, todos los archivos del mismo tipo van a tener la misma estructura el mismo formato.

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

F13) Tipos de archivos
En función de su contenido y su forma: podemos pensar en tres grandes grupos Texto Datos(registros) y Ejecutables.
Texto: archivos con únicamente texto, sin estructura interna.
Datos(registros): archivos de datos, que tienen una estructura interna, tienen un conjunto de registros que les permite almacenar esos datos.
Ejecutables: archivos que pueden ser ejecutados, osea que pueden dar lugar a procesos.

En la tabla tenemos algunos tipos de archivos con la extensión más común, y cual es la función que corresponde a ese tipo de archivo.
Por ejemplo los archivos ejecutables suelen tener la extensión "exe", "com", "bin" o podría no tenerlo en unix uno podría tener un archivo ejecutable que no tenga extensión. La función que tienen es justamente estar listos para ser ejecutados dentro del sistema de computación.
Podría ser un archivo objeto, suele tener extensión "obj" o "o". Es un archivo que no está linkeado, está compilado para que el sistema pueda linkearlo y pueda cargalo dinamicamente.
Lo mismo para archivos de codigo fuente (Source code), batch, de texto ("txt" o "doc"), archivos de procesador de texto, librerias ("lib", "dll"), de tipo archivo ("arc", "zip" o "rar") que comprimen archivos, también archivos multimedia ("mov", "mp3", etc).
Todos estos son tipos de archivos y la extensión usual que generalmente viene determinada por la aplicación que manipula ese tipo de archivo.

F14) Atributos de Archivos
Todos los archivos tienen atributos, así como los procesos tienen atributos que se guardan en su PCB. A diferencia de los atributos de los procesos que son volátiles (existen mientras exista el proceso), los atributos de los archivos son persistentes, tanto como los archivos. Si el archivo persiste también deben hacerlo sus atributos. Cuando existe un archivo hay un conjunto de atributos del archivo que se mantienen en forma persistente junto con le archivo.
Cada SO determina cuales son los atributos que va a utilizar para un archivo, pero generalmente se determinan los siguientes:
Nombre // Atributo clave del archivo

Identifier-unique tag(número) que lo identifica en el file system.
Tipo (en SO que soportan tipos).
Puntero a la ubicación. //Necesitamos saber donde comienza físicamente el archivo.
Tamaño. // es decir, cuanto espacio ocupa.
Timestamp de creación. //fecha y hora de creación del archivo.
Propietario. // Quien es el propietario, porque en función del propietario podemos discriminar que puede hacer cada uno.
Protección (mecanismos para discriminar el acceso a lectura, escritura o ejecución). //Cuales son los indicadores de protección, ¿Quién puede leer escribir o ejecutar ese archivo?
Estos son solo algunos de los atributos cada SO almacena un conjunto de atributos de un archivo. Podemos verlo por ejemplo en windows haciendo click derecho sobre el nombre de un archivo estos atributos nos agregan información de gestión respecto del archivo. Es meta información administrativa burocrática que nos permite gestionar el archivo. Más allá de la información propiamente dicha contenida dentro del archivo.

F15) Operaciones sobre archivos
Otro aspecto fundamental en las convenciones de diseño de un sistema de archivo, tiene que ver con el conjunto de operaciones que vamos a poder realizar sobre los archivos.
El archivos es un "abstract data type" //Tipo de datos abstracto, por ende una estructura que ademas del contenido de la información dentro de esa estructura, tiene un conjunto de operaciones que se pueden realizar sobre esta estructura.
Create // Crear un archivo
Read
Write
Execute
Seek
Append // para agregar información al archivo.
Delete
Truncate// para cortar un archivo
Open
Close
Lock // para bloquearlo, es decir, para garantizar que solo ese proceso está manipulando ese archivo en un momento determinado. De esta manera evita conflictos de concurrencia.

//Definir entonces el set de operaciones que el SO va a permitir sobre los archivos es un aspecto fundamental, cada una de estas operaciones que manipulan archivos son instrucciones privilegiadas. Cada vez que un proceso de usuario quiera llevar adelante alguna de estas operaciones puestas a disposición por el SO, deberá hacerlo mediante una llamada al sistema.

El sistema operativo pone a disposición del usuario, módulos o programas del sistema, para que el usuario pueda realizar algunas operaciones sin ningún otro intermediario.

F16) Organización lógica de los archivos

Un aspecto fundamental que tiene que ver con los sistemas de archivos, es la organización lógica de los archivos

Es la forma en que los archivos son organizados lógicamente por el usuario. //El sistema operativo brinda al usuario una imagen ficticia de lo que es el hardware, una imagen ficticia de lo que es la información guardada en los dispositivos. Es el usuario el que decide como organiza la información dentro del dispositivo. Y lo hace desde una organización lógica lo hace en función de una estructura organizativa...

Determina la estructura organizativa con que el usuario dispone los archivos en los dispositivos. //Ordena los archivos en base a una estructura organizativa que el decide bajo determinadas pautas, que también deben definirse durante el tiempo de diseño, esas pautas forman parte del file system en sentido amplio. Y tiene que ver con la organización lógica de los archivos.

Considera dos tipos de entidades:

Directorios (o "carpetas" contenedoras de archivos) //Estructura de organización, son las carpetas las que nos permiten organizar la información contenida dentro de los archivos.

Archivos propiamente dichos. // Estructura de soporte de la información propiamente dicha

F17) Directorios de archivos

Estos directorios de archivos son el mecanismo de organización y contienen información sobre los archivos:

- Atributos.
- Ubicación.
- Propietario.

El directorio es en si mismo un archivo, que contiene información administrativa o burocrática, y es propiedad del SO. //Tenemos entonces archivos con información propiamente dicha, y tenemos también directorios que almacenan la información burocrática de esos archivos. Información que permite al SO administrar el archivo (ubicarlo, controlar el acceso). Además de contribuir a la organización lógica contribuyen a la administración de la información del sistema de archivo.

Permite el mapping (mapeo) entre los nombres de archivo (identificación unívoca de los mismos) y los archivos propiamente dichos.

F18) Directorio de archivos

En términos de estructura

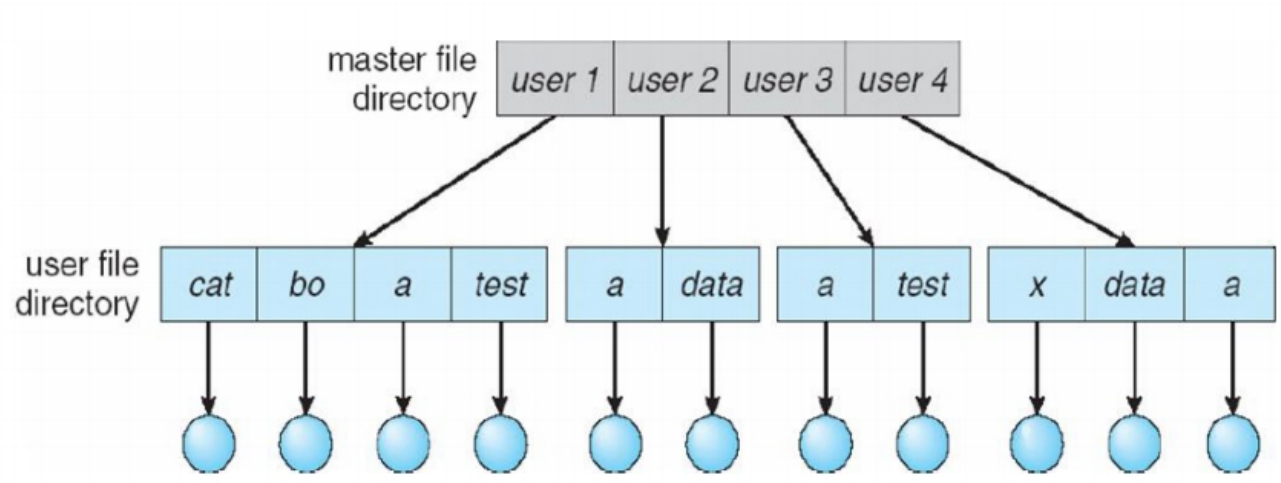
Cada directorio de usuario es una simple lista de entrada a los archivos del usuario. // Es decir, tenemos un directorio por usuario y cada usuario tiene en ese directorio la lista de entradas a los archivos de ese usuario.

Puede representarse con un simple archivo secuencial, donde el nombre de cada archivo es su clave unívoca (File-Id).

Hay un directorio maestro general y uno para cada usuario. El directorio maestro general contiene una entrada para cada directorio maestro de usuario, con su información de acceso

(privilegios). //Es decir, los privilegios que se requieren para que un usuario pueda acceder a ese directorio maestro de ese usuario (podría ser el propietario o bien otros usuarios que estén habilitados para acceder a ese directorio maestro de otro usuario).
No conoce de la organización física de los archivos. //Porque la organización de los archivos es lógica y la reconoce como lógica sin detenerse en como están físicamente alojados (eso más bien tiene que ver con el método de asignación de archivos dentro del dispositivo de hardware).

F19) Directorio de archivos
//En este gráfico se puede ver un ejemplo de una estructura donde tenemos un directorio de archivo maestro dividido por usuarios, usuario 1 , 2 , 3 y 4. De cada uno de ellos sale al directorio de archivo de usuario. En este directorio tenemos la organización lógica de los archivos (carpetas) y además los nodos u hojas que son los archivos propiamente dichos.



F20) Directorio tipo árbol
//En cuanto a la estructura de estos directorios el esquema más comúnmente difundido en los SO modernos es el de directorio de tipo árbol. Es decir:
Existe un directorio maestro que contiene un número determinado de directorios de usuario. //Y de
Cada uno de estos directorios puede tener, a su vez subdirectorios y archivos como entradas.
//Esto significa que puedo tener un esquema de árbol donde tengo directorios que son padres de directorios y algún directorio que sea padre tanto de directorios como de archivos. Y en cada caso en estos directorios estarán las respectivas entradas.
Las hojas del árbol (es decir, el último nivel del árbol) son los archivos //propiamente dichos que contienen la información propiamente dicha y no directorios que constituyen información burocrática.

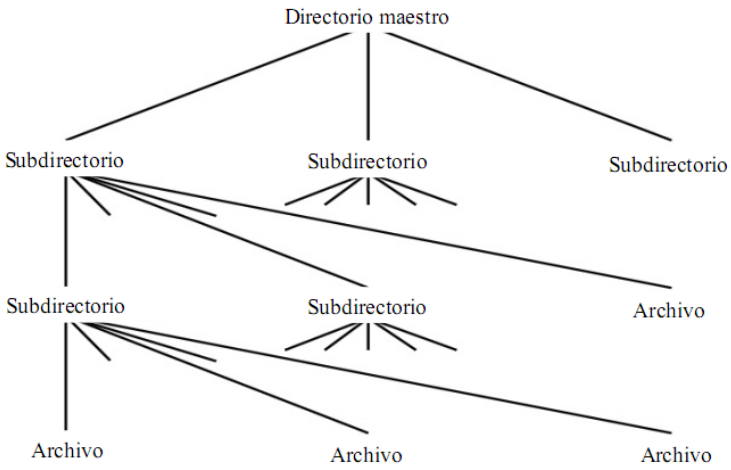
F21) Directorio tipo árbol
//En este esquema de directorios de tipo árbol
Cualquier archivo puede ser localizado siguiendo un camino desde el directorio raíz o maestro, descendiendo por varias ramas: // que conforman el camino para llegar hasta el archivo.
Nombre de camino del archivo (Pathname).

Se pueden tener varios archivos con el mismo nombre de archivo mientras tengan diferentes nombres de camino. //Puedo tener más de un archivo que se llame "práctico1" siempre y cuando los distintos archivos llamados "práctico1" estén en distintas ramas del árbol.
El directorio actual es el directorio de trabajo.

Las referencias a los archivos son relativas al directorio de trabajo. //Esto significa que si estoy parado en el directorio "sistemas operativos" la referencia al archivo "practico1" significa que me estoy refiriendo al "practico1" de "sistemas operativos".

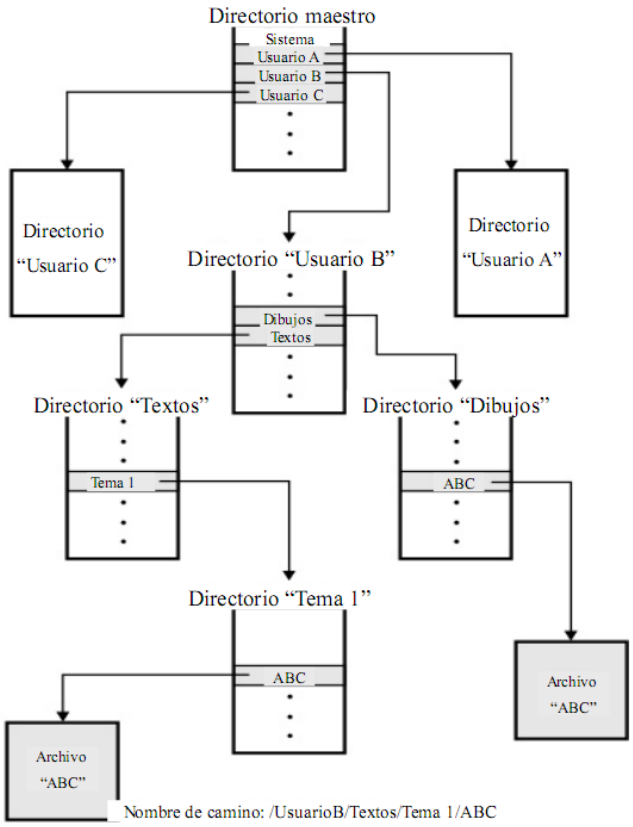
F22) Directorio tipo árbol

//En esta figura de stalling pueden ver como un directorio maestro tiene subdirectorios, y estos subdirectorios tienen a su vez subdirectorio y algunos archivos. Y cada subdirectorio puede tener finalmente archivos.



F23) Directorio tipo árbol

//Acá lo que vemos es la estructura de gestión de este directorio tipo árbol. En la raíz del directorio maestro (en la raíz del árbol) se referencia a los distintos usuarios del sistema, es decir donde tengo puntero a la raíz de los directorios de cada uno de los usuarios del sistema.
Por ejemplo el puntero "Usuario B" nos llevará al directorio del usuario B, dentro de ese directorio del usuario B vemos que tenemos dos carpetas "Dibujos" y "Textos"



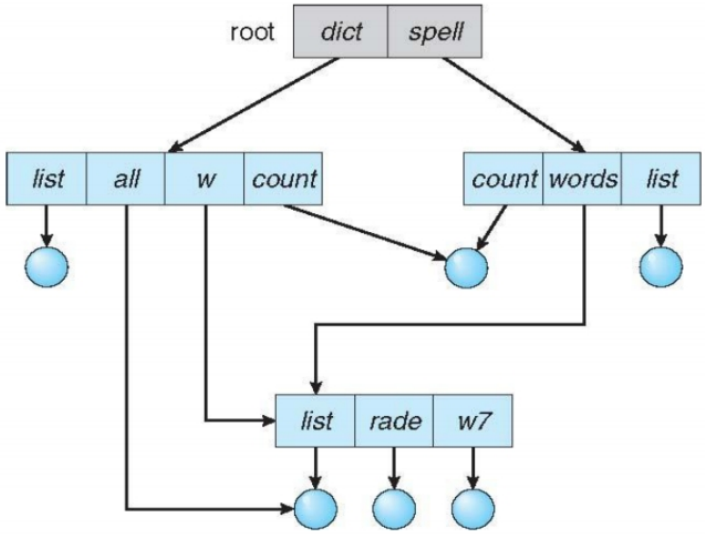
/// // texto sobre pathname y eso, referencia absoluta y relativa y eso ...

F24) Directorio grafo acíclico

Una evolución dentro de este tipo de directorios, es la evolución de un árbol a un grafo acíclico.
Se trata de un modelo de árbol donde un nodo del mismo archivo puede tener más de un padre.
A estos nodos se puede llegar por más de un camino, por medio de "atajos" o links. //estos "atajos" son los que me permiten acceder al archivo propiatydich.
Se permite que los links vayan solo a archivos (no a directorio), para garantizar que sea acíclico y evitar posibles "islas". //es decir, referencias circulares.
//De esta forma si nosotros permitimos que desde una carpeta se pueda acceder a un archivo que está en otra carpeta tendremos la posibilidad de llegar rápidamente a ese archivo desde más de un lugar por medio justamente de un atajo o link.

F25) Directorio grafo acíclico

Aquí en este gráfico de silvershark, tenemos un grafo acíclico donde en la raíz se almacenan dos punteros, uno a cada directorio respectivamente. El directorio de "dict" y el de "spell", ambos directorios pueden acceder al mismo archivo "count". Desde cualquiera de los dos directorios puedo acceder al mismo archivo solo por medio de su nombre.



F26) Directorio grafo acíclico

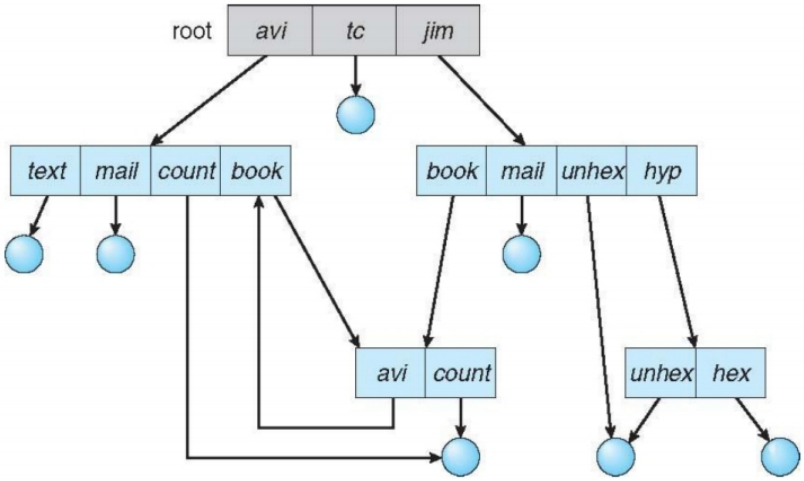
Estos links o atajos que nos permiten llegar a un archivo de forma rápida son de dos tipos. Por un lado los soft link, o links livianos, conocidos como accedo directo.
Hay dos tipos de links:
Soft link (acceso directo): el nodo es apuntado por un puntero dentro de la estructura de la tabla de alocación de cada padre, como si fuera un archivo propio. //el archivo va a estar referenciado desde más de un directorio.
Cuando el archivo es borrado por alguno de sus padres, es eliminado materialmente, y el link que lo apuntaba desde el otro padre queda apuntando "al vacío". //Ahí tendremos entonces un acceso directo a un archivo que no existe.

F27) Directorio grafo acíclico.

Hard link: el nodo es apuntado por un puntero dentro de la estructura de la tabla de alocación de cada, como si fuera un archivo propio. Cuando el archivo apuntado por el hard link es "borrado", se lo "desengancha" del padre que lo eliminó pero sigue existiendo para el otro padre. Requiere de un Contador de padres. Cuando el Contador de padres llega a cero, entonces se borra materialmente el archivo.
//En hard link en cambio si bien la referencia es la misma se agrega un atributo que tiene que ver con la cantidad de padres,

F28) Directorio grafo general

Esta es la siguiente evolución dentro de los distintos esquemas pasamos del árbol a un grafo acíclico y ahora pasamos a un grafo general. Donde inclusive están permitidos los ciclos, es decir, está permitido que un directorio apunte o tenga como hijo a otro directorio. Se puede ver en la figura que el directorio book y el directorio avi conforman un ciclo.



En los SO que tienen este esquema por lo general se requiere un "Garbage Collection" que lo que hace es barrer todas las estructuras de directorios buscando justamente estos ciclos que no son apuntados por nadie estas "islas". O Conjunto de directorios que se apuntan mutuamente y que no pueden ser accedidos desde la raiz

F29) Archivos compartidos

En términos de acceso a los archivos un aspecto importante tiene que ver con la posibilidad de compartirlos.

En un sistema multiusuario, existe la necesidad de permitir a los usuarios compartir archivos.

Dos cuestiones:

- Los derechos de acceso. //es decir, quien puede acceder a esos archivos y que puede hacer.
- La gestión de los accesos simultáneos. //es decir, como controlar y garantizar que no hay dos procesos que están escribiendo simultáneamente un mismo archivo. De forma tal que lo que un usuario escribe otro usuario lo sobrescribe, en definitiva no se garantiza la unicidad o la exclusión mutua de este tipo de operaciones.

F30) Derechos de acceso

Cuando hablamos de derecho de acceso lo primero que hay que tener en cuenta es que
Cada archivo tiene un propietario.
El propietario dispone de todos los derechos de acceso y manipulación sobre el archivo.
Puede, además, otorgar derechos a los otros usando las siguientes clases de usuarios:

Usuario específico. // otorgar derechos a un usuario en particular
Grupo de usuarios.
Todos los usuarios (archivos públicos).

F31) Acceso simultáneo a los archivos

El usuario puede:

- Bloquear el archivo entero cuando lo vaya a actualizar. //El usuario bloquea al archivo para usarlo él (sea para escritura, lectura) y que ningún otro usuario pueda modificarlo. Cuando termina de usarlo lo desbloquea. Lo libera y lo cierra, para que cualquier otro usuario pueda

utilizarlo.

- Bloquear los registros individuales que va a actualizar (preferible). //Otra opción es bloquear únicamente registros individuales, esta es una opción preferible, porque permite que más de un usuario estén trabajando sobre el mismo archivo garantizando que ambos procesos no están trabajando sobre los mismos registros.

Al diseñar la posibilidad de accesos compartidos, deben abordarse aspectos de exclusión mutua e interbloqueo.

F32) Modos de acceso a los archivos

Otro punto fundamental otra convención importante que tenemos que tener en cuenta a la hora de diseñar nuestro sistema de archivos, son los modos de acceso que vamos a permitir a los archivos. Generalmente hablamos de 3 modos de acceso, el directo, el secuencial y el indexado.

Directo: el acceso a un bloque del archivo se hace en forma directa, sin necesidad de haber accedido a los anteriores para lograrlo. //Se puede acceder al bloque numero 15 sin haber accedido a los anteriores 14.

Secuencial: el acceso a un bloque del archivo se hace en forma secuencial, requiriendo indefectiblemente haber accedido a los anteriores para lograrlo. //En este caso si es necesario acceder a los 14 bloques anteriores antes de acceder al bloque 15.

Indexado: el acceso a un bloque del archivo se hace mediante un índice que nos lleva al mismo. // Una opción intermedia es el indexado// el acceso a un bloque del archivo se hace mediante un índice que nos lleva al mismo, el acceso no es directo ni secuencial. Accedo a un índice, el índice apunta directamente al bloque del archivo que deseo acceder. Como mínimo tengo que hacer 2 accesos para acceder a un bloque con acceso indexado.

F33) Alocación de archivos

Otro aspecto fundamental en términos de convenciones de diseño. Tiene que ver con la forma en la que vamos a alocar o asignar los archivos. //Alocación o asignación de archivos.

El sistema de Gestión de Archivos debe asignar espacio a los archivos (organización física). //Es el que se encarga de la organización física. Ya no nos importa la organización lógica sino que nos importa como vamos a alojarlos.

-El archivos es un conjunto de bloques en el dispositivo.

-Requiere que se declare el tamaño máximo (cantidad de bloques) del archivo al momento de crearlo.

-Es difícil estimar el posible tamaño del archivo. Se tiende a sobrestimarlo para que no le falte espacio.

El sistema de Gestión de Archivos debe conocer el espacio disponible (bloques) para asignar a los archivos (gestión de espacio libres).

F34) Métodos de asignación: CONTIGUA

Existen varios métodos de asignación de archivos en un dispositivo de entrada salida.

El método de asignación CONTIGUA

Cuando se crea un archivo se le asigna un único conjunto contiguo de bloques.

//Si se quisiera alojar un archivo de 4 bloques, debería encontrar 4 bloques contiguos libres dentro del dispositivo para poder guardar ahí los 4 bloques.

La tabla de asignación necesita sólo una entrada por cada archivo: //Cuando es CONTIGUA solo necesitamos una entrada por archivo.

- Bloque de comienzo y longitud del archivo. //Vamos a tener un puntero al comienzo del archivo y la cantidad de bloques que componen al archivo.

Se produce fragmentación interna en el último bloque. //Siempre en todos los casos el último bloque puede no estar completo.

Se produce fragmentación externa (bloques libres entre archivos) => Compactación //(se juntan los bloques ocupados por un lado y los bloques libres por otro), este proceso de compactación implica reubicar los archivos. Tarea larga compleja, impredecible, no se puede interrumpir.

//Este tipo de fragmentación que requiere de compactación para poder trabajar hace que sea un tipo de asignación poco conveniente para sistemas de tiempo real.

Si se daña un bloque, se pierde solo ese. //la información dentro del bloque se pierde, pero no se pierde todo el archivo. Se puede seguir trabajando con el archivo, solo se perdió la información en el bloque dañado.

F35) Método de asignación: CONTIGUA

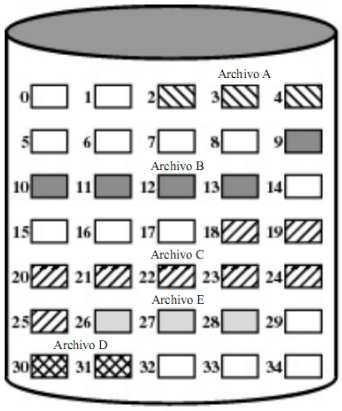


Tabla de asignación de archivos

Nombre de archivo	Bloque de inicio	Longitud
Archivo A	2	3
Archivo B	9	5
Archivo C	18	8
Archivo D	30	2
Archivo E	26	3

En este gráfico de stalling tienen un dispositivo, en este caso un disco donde pueden ver que ese dispositivo está estructurado o físicamente organizado en un conjunto de bloques físicos consecutivos contiguos que van desde el 0 al 34. Cada bloque está identificado por un número. Si miramos la tabla de asignación de archivo en el directorio, vemos que en la carpeta hay un archivo A, B, C, D y E. El archivo A comienza en el bloque 2 y tiene una longitud de 3 bloques.

Si quisiéramos alojar un archivos de tamaño de 6 bloques, no se podría porque no hay 6 bloques contiguos disponibles, a pesar de tener en total 14 bloques libres no contiguos.

F36) Método de asignación: CONTIGUA

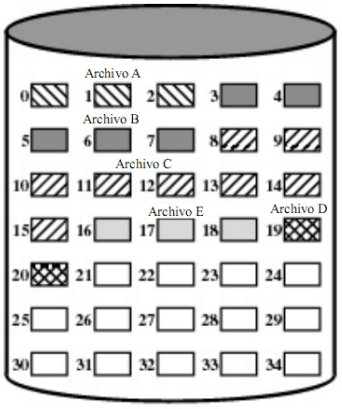


Tabla de asignación de archivos

Nombre de archivo	Bloque de inicio	Longitud
Archivo A	0	3
Archivo B	3	5
Archivo C	8	8
Archivo D	19	2
Archivo E	16	3

En esta otra imagen luego de la compactación, ahora los archivos están juntos y consecutivos sin huecos entre ellos, y además tenemos los bloques libres juntos al final del dispositivo. Esto nos permitiría alojar cualquier archivo con un tamaño igual al disponible.

El problema con este método contiguo es que si el archivo A fuera un archivo de datos que tiende a crecer, entonces el archivos A no podría crecer sobre el bloque 3 que es el que sigue, porque este bloque

está asignado a otro archivo. En este caso se podría realocar el archivo A en el bloque 21. Por lo cual el costo que tiene esta gestión contigua puede ser muy caro porque implica la lectura y la escritura de todos los bloques del archivo para poder trasladarlo a otro lugar.

F37) Método de asignación: VINCULADA

Un segundo método de asignación que evita los inconvenientes que tiene la asignación contigua es el método de asignación vinculada.

La asignación se hace con bloques individuales (no necesariamente contiguos) que se encadenan.

Cada bloque contiene un puntero al siguiente bloque de la cadena. //que conforman al archivo, un archivo entonces es una cadena de bloques, enlazados entre sí por medio de punteros.

En cada bloque hay información propiamente dicha //Información del archivo

e información administrativa (burocrática). //el puntero al siguiente bloque

La tabla de asignación necesita una sola entrada por cada archivo:

- Bloque de comienzo y longitud del archivo (cantidad de bloques).

F38) Método de asignación: VINCULADA

Como aquí todos los bloques que están disponibles están juntos.

No genera fragmentación externa. No requiere de compactación (aunque ayuda). //Mi archivo puede crecer usando cualquier bloque libre, sin necesidad de reubicarlo.

Se ajusta mejor para alojar archivos secuenciales.//es decir, archivos de acceso secuencial, donde para acceder al bloque 15 es necesario pasar por los 14 bloques anteriores. Indefectiblemente tengo que recorrer la cadena de bloque para llegar al bloque deseado.

Si se pierde un bloque, se pierde el resto del archivo luego de este (lost chain //o "cadena perdida").

Se puede mejorar la confiabilidad con doble cadena de punteros (en ambos sentidos). //Es decir, que cada bloque tenga dos punteros, un puntero al bloque siguiente y uno al bloque anterior. De esta forma tenemos una lista doblemente vinculada.

- En este caso, la tabla de asignación necesita dos entradas por cada archivo (Bloque de comienzo y Bloque de fin, longitud del archivo).
- Si se daña un bloque se pierde solo ese. // Dado que si un bloque se daña, podemos reconstruir el resto del archivo sin el bloque dañado. Si se pierden dos bloques el 4 y el 50 perderé entonces todos los bloques entre el 4 y el 50. Si fuera una asignación vinculada a cadena simple sería lo mismo, si fuera una asignación contigua solo se hubieran perdido el 4 y el 50.

Esta opción con un segundo puntero, mejora la confiabilidad pero también agrega mas información burocrática, implicando un mayor espacio destinado a guardar punteros. El método de asignación Vinculada ya ocupa más espacio que la contigua. Porque aloja la misma información pero además los punteros. Por eso el tamaño físico de un archivos en la asignación vinculada será mayor que en la asignación contigua.

F39) Método de asignación: VINCULADA

En este gráfico de stalling se puede ver como el archivo B tiene al bloque 1 como bloque de inicio y tiene 5 de longitud. El bloque 1 tiene un puntero al bloque 8, del bloque 8 sigue el bloque 3, del 3 salta al 14 y del 14 al 28.

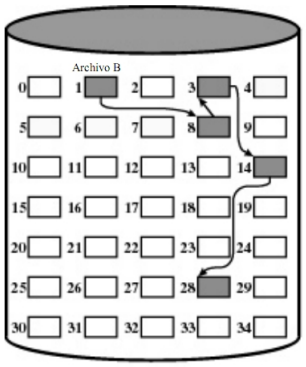


Tabla de asignación de archivos

Nombre de archivo	Bloque de inicio	Longitud
•••	•••	•••
Archivo B	1	5
•••	•••	•••

F40) Método de asignación: VINCULADA

En este esquema nos encontramos también con la asignación Vinculada, es el mismo archivo B, pero luego de haber sido compactado. Ahora el archivo está alojado en bloques contiguos, pero a ellos no se accede de forma directa, sino que indefectiblemente se accederá por medio de la cadena de punteros.

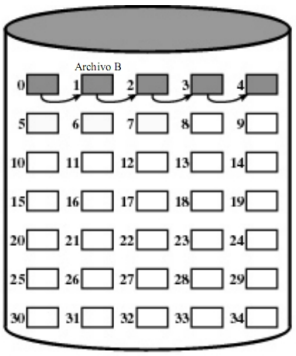


Tabla de asignación de archivos

Nombre de archivo	Bloque de inicio	Longitud
•••	•••	•••
Archivo B	0	5
•••	•••	•••

//El problema de la asignación vinculada es que generá una cadena de muchos bloques para llegar a un bloque determinado, para llegar al bloque 80 tengo que leer 80 bloques. Tiene un tiempo de acceso muy largo para acceder y leer un bloque. Además es un tiempo impredecible porque depende de la posición del bloque.

F41) Métodos de asignación: INDEXADA

La asignación indexada es un método que nos permite optimizar ese problema.

Tenemos aquí dos tipos de bloques:

-Bloques de índices (contienen punteros, información burocrática). //No hay información propiamente dicha.

-Bloques de datos (información propiamente dicha del archivo). //Bloques muy similares a los de la asignación contigua, solo tienen información y nada de burocracia.

La tabla de asignación de archivos contiene un puntero al primero bloque de índices del archivo.

F42) Métodos de asignación: INDEXADA

Los bloques de índices contienen N punteros ordenados a bloques de datos del archivo.

Los bloques de índices están encadenados entre sí. El ultimo puntero de un bloque de índices apunta a otro bloque índice (con otros N punteros a bloques de datos). //Los bloques de índices son a su vez una lista vinculada de bloques de índices.

No hay fragmentación externa. //al igual que en la asignación vinculada

/*El acceso no es directo y tampoco es secuencial decimos que:*/

El acceso es pseudo-directo: secuencial a la lista de bloques índices y directo a los bloques de datos.

F43) Métodos de asignación: INDEXADA

En este gráfico de stalling vemos también un ejemplo de asignación indexada, donde el archivo B empieza en

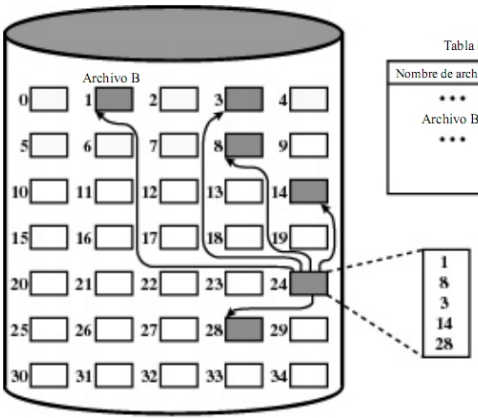


Tabla de asignación de archivos

Nombre de archivo	Bloque índice
•••	•••
Archivo B	24
•••	•••

la dirección 24, en la dirección 24 está el bloque índice del archivo B. En él se almacena la información burocrática, se almacenan los punteros a los bloques de datos. En el ejemplo esos punteros son (1, 8, 3, 14, 28)

F44) Métodos de asignación: UNIX

Estos tres métodos de asignación (Contigua, vinculada e indexada) son los métodos clásicos tradicionales o básicos a partir de los cuales se van generando distintas variaciones. A modo de ejemplo veremos el caso de UNIX.

UNIX propone un nuevo método de asignación especifico de este sistema operativo.

UNIX utiliza una variación de la asignación vinculada que evita el acceso secuencial a la cadena de bloques de índices: //El defecto que tenía la vinculada que era la cadena de bloques índices que cuando fuera muy grande el archivo sería una cadena larga. Implementa una variación que evita esa cadena secuencial.

Lo implementa con un conjunto de entradas dentro de su directorio, que son

bloques de índice de primer nivel, que contienen punteros directos a bloques de datos,

bloques de índice de segundo nivel que contienen punteros a bloques de índice, que a su vez contienen punteros a bloques de datos.

Y bloques de 3er nivel que al igual que en el caso anterior agrega un nivel mas, es decir, Bloque de indice que apunta a bloques de índices, esos bloques apuntan nuevamente a bloques de índices y esos bloques de índices tienen punteros que apuntan finalmente a bloques de datos.

Entonces hay 3 tipos de bloques de índices, de primer segundo y tercer nivel. Y obviamente tengo los bloques de datos es decir aquellos que tienen la información propiamente dicha.

Hay entonces 4 tipos de bloques. 3 para información estrictamente burocrática y uno para bloques de datos.

F45) Métodos de asignación: UNIX

Los bloques de índices de 2do y 3er nivel disminuyen exponencialmente el tiempo de acceso a un bloque de datos, al no tener que recorrer secuencialmente como en la indexada pura (que puede ser una cadena muy larga en archivos grandes). //En el peor de los casos el bloque de datos buscado está en el 3er nivel de indirección. Cualquiera sea el tamaño del archivo y cualquiera sea el bloque de datos dentro del archivo.

No hay fragmentación externa.

El acceso es pseudo-directo, pero mucho más rápido: secuencial a los bloques índices (a lo sumo 3 accesos) y directo a los bloques de datos.

F46) Métodos de asignación: UNIX

La tabla de asignación de archivo contiene una estructura de acceso para cada archivo denominada i-nodo que contiene 12 punteros:

-9 punteros a bloques de datos. Cada puntero señala a un bloque de datos puro y garantiza que el acceso a los primeros bloques de datos del archivo sea directo. //Tengo 9

punteros a los 9 primeros bloques del archivo en el directorio, para estos 9 primero bloques no es necesario pasar por ningún bloque índice, se accede de forma directa desde el directorio. Si la mayoría de los archivos son pequeños ... es bueno para los archivos de poco tamaño.
//El problema que tiene la asignación indexada que para archivos pequeños necesito como mínimo un bloque de índice, estoy agregando una burocracia del 100%, (Para un archivo de un bloque de información tengo que leer un bloque de índice). Con estos 9 punteros en el directorio asegura un acceso directo para los 9 primero bloques de datos del archivo.

-1 puntero a un bloque de índices de 1er nivel, cuyos punteros apuntan directamente a bloques de datos.

-1 puntero a un bloque de índices de 2do nivel, cuyos punteros apuntan a bloques de índices de 1er nivel, cuyos punteros apuntan directamente a bloques de datos.

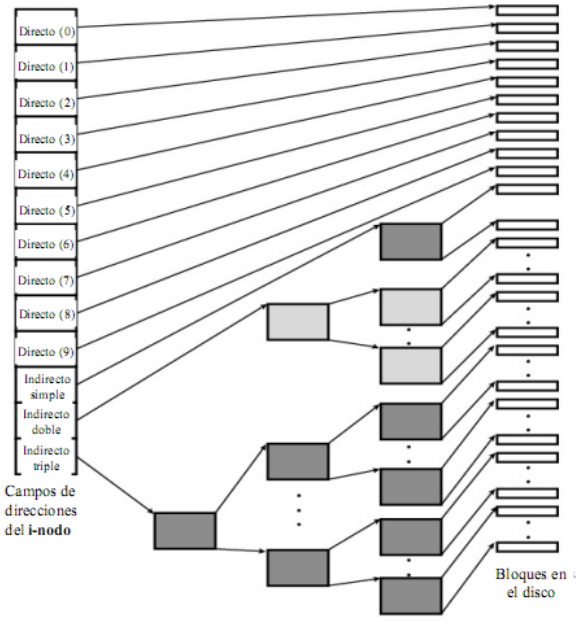
-1 puntero a un bloque de índices de 3er nivel...

F47) Métodos de asignación: UNIX

Asignación en UNIX (i-nodos)

Espero que en esta figura también de stalling, los primeros 10 punteros de esta estructura vertical que tienen a la izquierda de la pantalla son campos de dirección del inodo, esa estructura se llama i-nodo es la estructura fundamental que se encuentra en el directorio de UNIX una para archivo.

En este caso los primeros punteros que van del 0 al 9, son los punteros directos, que apuntan a bloques de datos. El primer bloque indirecto simple es un puntero, a otro bloque de índices que apuntan nuevamente a bloques de datos ...



Asignación en UNIX (i-nodos)

Con esto terminamos la forma en la que alojamos los archivos sabemos como se guardan, sabemos como están físicamente mapeados, en el disco.

F48) Gestión del espacio libre: estructuras

A la hora de guardar esos archivos, es necesario disponer de bloques libres, donde voy a guardar la información que contienen los bloques que quiero guardar y que conforman el nuevo archivo que estoy creando.

Para la gestión del espacio libre, el SO requiere de estructuras de soporte. Las más conocidas:

- Listas de libres (Free List)
- Lista de principio y cuenta
- Mapa de bits (BitMap) //un bit por cada bloque

F49) Lista de Libres

La lista de libres (Free List) es una lista encadenada de bloques libres.

Cada bloque tiene un puntero al próximo bloque libre. //Como si fuera un gran archivo alojado en forma vinculada pero que tiene enlazados todos los bloques disponibles en el disco.

Cuando se libera un bloque se lo engancha al final de la cadena.

Cuando se requiere un bloque, se desengancha uno de la cadena. //Y es el que se le asigna al archivo que se esta creando.
Ideal para la asignación vinculada. //Porque trabaja con el mismo criterio. Esta estructura de espacios libres podríamos implementar y manejarla con el mismo manejador de archivos que maneja los archivos que contienen información. La lista de libres no sería más que un archivo en una estructura de asignación vinculada, donde los bloques que disponen este archivo son los bloques disponibles.
Malo para asignación contigua. //Porque tener enlazados todos los bloques libres

F50) Lista de Principio y Cuenta de libres
Precisamente si lo que tenemos es un mecanismo de asignación contigua y quiero manejarme con una lista de espacios libres una freeList, tendría que hacer una variación de esta estructura de datos, es lo que se llama Lista de principio y cuenta de libres.
Se trata de una lista encadenada de bloques libres donde cada bloque guarda la cantidad de bloques contiguos libres que tiene.
Es una lista que encadena a los primeros bloques de cada conjunto contiguo de libres (huecos) y conoce su tamaño. //Tengo una lista encadenada donde tengo en cada bloque de la lista tengo el puntero al siguiente como información administrativa, pero además guardo un segundo atributo que es la cantidad de bloques contiguos que tiene ese bloque. entonces se almacena, un puntero al siguiente bloque libre "el n° 8", y se almacena la cantidad de bloques libres contiguos "tiene 4 bloques libres contiguos". Significaría que los bloques 8,9,10 y 11 están libres y contiguos.
//Esto se puede usar para una asignación vinculada tradicional como para un asignación contigua
//Si deseara alojar un archivo que necesita 7 bloques libres contiguos, debería recorrer la lista de libres preguntando por la cantidad de bloques contiguos del siguiente bloque, hasta encontrar una que tenga al menos 7.
Es más corta que la Free List //Tiene menos elementos porque no están todos los bloques encadenados, sino que cuando tengo 8 libres encadeno solo el primero y no los otros 7. Por lo tanto la lista es más corta y mas eficiente su uso.
Ideal para asignación contigua. Sirve para vinculada.

F51) BitMap
//La tercera opción de implementación es la que llamamos bitmap o mapa de bits.
Se trata de un arreglo unidimensional de bits con tantos elementos como bloques en el dispositivo.
//Si el dispositivo tiene 25 bloques en total, entonces necesito un arreglo con 25 elementos tipo bit.
Cada elemento contiene un valor binario que indica si ese bloque está ocupado o libre.
//$BitMap_8$ hace referencia al bloque 8 del dispositivo, si hay un 0 entonces el bloque está libre, si en cambio hay un 1 entonces el bloque estará ocupado. //El vector está cargado en memoria, es barato.
Es más económica que la FreeList y de rápido acceso.
Ideal para asignación contigua. // Porque ahora buscar una secuencia contigua de 8 bloques es algo que se puede hacer en el vector cargado en memoria.

Sirve también para asignación vinculada. // y también para indexada.
//Es una estructura de datos barata de bajo costo, muy performante que nos sirve para cualquier método de alocación.

F52) Gestión del espacio libre: huecos
Por ultimo un comentario que tiene que ver sobre la gestión del espacio libre, y que tiene que ver con los huecos.
Al conjunto de bloques disponibles contiguos se lo llama "**Fit**" o "**hueco**"
Para la selección del espacio libre (hueco) para asignar un archivo, existen 3 estrategias o criterios:

- **First Fit** (Primer hueco, más rápido) //El problema con este criterio, es que puede contribuir a tener una mayor fragmentación. //Si estoy en un sistema de tiempo real que me interesa tener una respuesta rápida, sin importar la fragmentación entonces, FirstFit sería una buena estrategia.
- **Best Fit** (Mejor hueco, menos fragmentación externa, ideal para archivo inmutables) //Esto genera la menor fragmentación externa posible. Si guardo en un cd de solo lectura, o si guardo un archivo ejecutable que tiene muy baja posibilidad de cambiar en su tamaño, entonces best fit es la mejor estrategia.
- **Worst Fit** (Peor hueco, mas fragmentación externa y chances de crecimiento para los archivos sin necesidad de realocarlos) //Para archivos que tienden a crecer como un archivo de datos de una colección de personas, sería conveniente esta estrategia para evitar reubicar un archivo grande.

sadsadsaasdsad